

Data Structures in C++:

Linked Lists, Dynamic Arrays, and Amortized Analysis

1 Linked Lists

A linked list is a linear data structure where elements are not stored in contiguous memory locations. Instead, each element (called a "node") contains two parts: the data itself, and a pointer (or reference) to the next node in the sequence.

In C++, you can handle linked lists in two ways: using the standard library, or building your own from scratch.

1.1 1. The Standard Library Way (`std::list` / `std::forward_list`)

Modern C++ provides two built-in containers for linked lists. You should default to these in production code unless you have a specific reason to build your own.

- `std::list`: A doubly-linked list (nodes have pointers to both the next and previous nodes).
- `std::forward_list`: A singly-linked list (nodes only point to the next node, using less memory).

Here is how you use `std::list`:

```
1 #include <iostream>
2 #include <list>
3
4 int main() {
5     // Create a doubly-linked list of integers
6     std::list<int> sequence = {10, 20, 30};
7
8     // Insert at the front (O(1) time complexity)
9     sequence.push_front(5);
10
11    // Insert at the back (O(1) time complexity)
12    sequence.push_back(40);
13
14    // Iterate through the sequence and print
15    for (int value : sequence) {
16        std::cout << value << " -> ";
17    }
18    std::cout << "END\n";
19
20    return 0;
21 }
```

1.2 2. Custom Implementation (Singly Linked List)

If you are studying data structures or need strict control over memory, you build a custom sequence using a `struct` or `class`.

1.2.1 The Node Structure

The foundation of a linked list is the `Node`.

```
1 struct Node {
2     int data;           // The value stored in the node
3     Node* next;        // Pointer to the next node in the sequence
4 }
```

```

5 // Constructor to easily create new nodes
6 Node(int val) : data(val), next(nullptr) {}
7 };

```

1.2.2 The List Manager

To manage the sequence of nodes, you typically create a class that keeps track of the "head" (the very first node).

```

1 #include <iostream>
2
3 class LinkedList {
4 private:
5     Node* head; // Points to the first node
6
7 public:
8     LinkedList() : head(nullptr) {}
9
10    // Insert a new node at the front of the sequence
11    void insertFront(int val) {
12        Node* newNode = new Node(val);
13        newNode->next = head; // Point new node to current head
14        head = newNode;      // Make head point to the new node
15    }
16
17    // Print the entire sequence
18    void printSequence() {
19        Node* current = head;
20        while (current != nullptr) {
21            std::cout << current->data << " -> ";
22            current = current->next; // Move to the next node
23        }
24        std::cout << "nullptr\n";
25    }
26
27    // Clean up memory (Destructor)
28    ~LinkedList() {
29        Node* current = head;
30        while (current != nullptr) {
31            Node* nextNode = current->next;
32            delete current;
33            current = nextNode;
34        }
35    }
36 };
37
38 int main() {
39     LinkedList list;
40
41     list.insertFront(30);
42     list.insertFront(20);
43     list.insertFront(10);
44
45     // Output: 10 -> 20 -> 30 -> nullptr
46     list.printSequence();
47
48     return 0;
49 }

```

1.3 Why Implement Manually When We Have STL?

For 99% of modern C++ programming, writing a custom linked list is reinventing the wheel. The STL's `std::list`, `std::forward_list`, and `std::deque` are rigorously tested, memory-safe, and highly optimized.

The only reasons the manual implementation still gets focus are:

- **The Technical Interview Game:** Big Tech still obsesses over testing raw pointer manipulation. Detecting a cycle or reversing a list by physically rewiring pointers is a standard filter.

- **Cache Locality Intuition:** Building one from scratch forces you to see exactly *why* a `std::list` is usually slower to traverse than a `std::deque` or `std::vector` (nodes are scattered randomly in the heap, causing CPU cache misses).
- **Kernel & Embedded Systems:** In environments like OS kernel development or strict low-memory microcontrollers, the STL is often completely disabled or banned to prevent hidden allocation overhead.

When you are pushing through those 20-hour study stretches, tracking `head->next->next` pointers with a pen might feel tedious, but that manual tracking wires your brain to understand dynamic memory routing. It shows you exactly what the STL abstracts away.

2 Dynamic Array Sequence (`std::vector`)

A dynamic array sequence is an array that automatically resizes itself when it runs out of space. In the C++ STL, this is `std::vector`.

Unlike a linked list where nodes are scattered in memory, a vector guarantees **contiguous memory allocation**. Because the CPU fetches memory in chunks (cache lines), iterating through contiguous data results in massive performance gains (cache locality) compared to following pointers across the heap.

2.1 How it Actually Works Under the Hood

A `std::vector` secretly manages a standard, fixed-size array on the heap using three pointers (or integer tracking):

1. **Size:** The number of elements currently stored.
2. **Capacity:** The total number of elements the current underlying array can hold.
3. **Data:** A pointer to the first element in memory.

2.1.1 The Reallocation Mechanism

When you push an element but `size == capacity`, the vector cannot simply append to the end because adjacent memory might be occupied. Instead, it executes a costly resizing operation:

1. Allocates a brand new, larger array (usually 1.5x or 2x the old capacity).
2. Copies (or moves) all existing elements from the old array to the new one.
3. Deletes the old array.
4. Inserts the new element.

Because the capacity grows exponentially, reallocations become increasingly rare, making insertion at the end take **Amortized** $O(1)$ time.

```

1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     // Initializes with size 0, and a small default capacity
6     std::vector<int> nums;
7
8     // It's a good habit to reserve space if you know the maximum size in advance.
9     // This prevents the costly reallocation process entirely.
10    nums.reserve(100);
11
12    nums.push_back(10);
13    nums.push_back(20);
14    nums.push_back(30);
15
16    // O(1) random access, unlike linked lists
17    std::cout << "Second element: " << nums[1] << "\n";
18
19    // Removing from the end is O(1)

```

```

20     nums.pop_back();
21
22     std::cout << "Current Size: " << nums.size() << "\n";
23     std::cout << "Current Capacity: " << nums.capacity() << "\n";
24
25     return 0;
26 }

```

3 Amortized Analysis and Dynamic Array Deletion

Amortized analysis calculates the average time taken per operation over a sequence of operations, guaranteeing that even if a single operation is very expensive, the average cost remains small.

For dynamic arrays, insertion and deletion rely on this mathematical guarantee to achieve $O(1)$ amortized time.

3.1 Proving Insertion is Amortized $O(1)$

When a dynamic array hits capacity N , inserting the next element requires allocating a $2N$ array and copying all N elements. This specific insertion takes $O(N)$ time. However, because the array has doubled, the *next* N insertions will simply drop into empty slots, taking $O(1)$ time each.

Using the **Aggregate Method**, we can sum the total work for inserting N elements (assuming resizing happens at powers of 2):

1. **Cost of standard insertions:** 1 unit of work per element = N .

2. **Cost of resizing copies:** We copy elements at sizes 1, 2, 4, 8... up to N .

The sum of this geometric series is:

$$1 + 2 + 4 + \dots + N = 2N - 1$$

Total work for N elements = $N + (2N - 1) = 3N - 1$.

Amortized cost per element = $\frac{3N-1}{N} \approx 3$.

Since the average cost is bounded by a constant (3 operations), insertion is **Amortized $O(1)$** .

3.2 Dynamic Array Deletion

Deleting elements introduces a new problem: when should the array shrink its capacity to save memory?

3.2.1 The "Thrashing" Problem

A naive approach would be to shrink the capacity by half as soon as the array is exactly 50% full. Imagine an array with capacity $N = 16$ and size 16.

1. **Push:** Array doubles to capacity 32, copies 16 elements (Cost: $O(N)$). Size = 17.

2. **Pop:** Size drops to 16.

3. **Pop:** Size drops to 15. The array is < 50% full. It shrinks capacity to 16, copying 15 elements (Cost: $O(N)$).

4. **Push:** Size returns to 16.

5. **Push:** Array doubles to 32, copying 16 elements again (Cost: $O(N)$).

If you alternate push and pop at this specific boundary, **every single operation takes $O(N)$ time**. This catastrophic performance failure is called thrashing.

3.2.2 The 1/4 Shrink Rule

To maintain $O(1)$ amortized time for deletion, dynamic arrays use hysteresis (a buffer zone): **They only halve their capacity when they become 25% (1/4) full.**

By waiting until the array is 1/4 full to shrink to 1/2 capacity, the structure guarantees that after any resize (growing or shrinking), the array is exactly half full. This means it must undergo at least $N/2$ cheap $O(1)$ operations before it could possibly need to resize again, successfully spreading the $O(N)$ cost and restoring Amortized $O(1)$ performance.